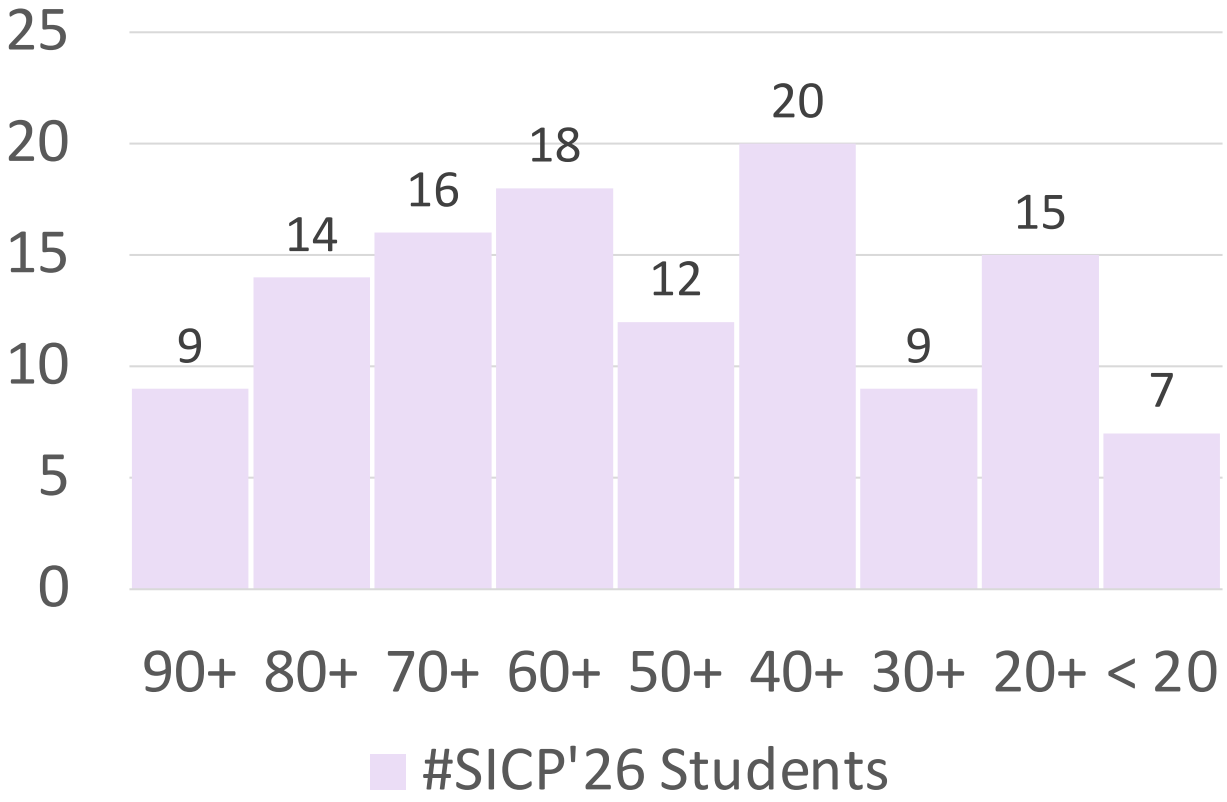
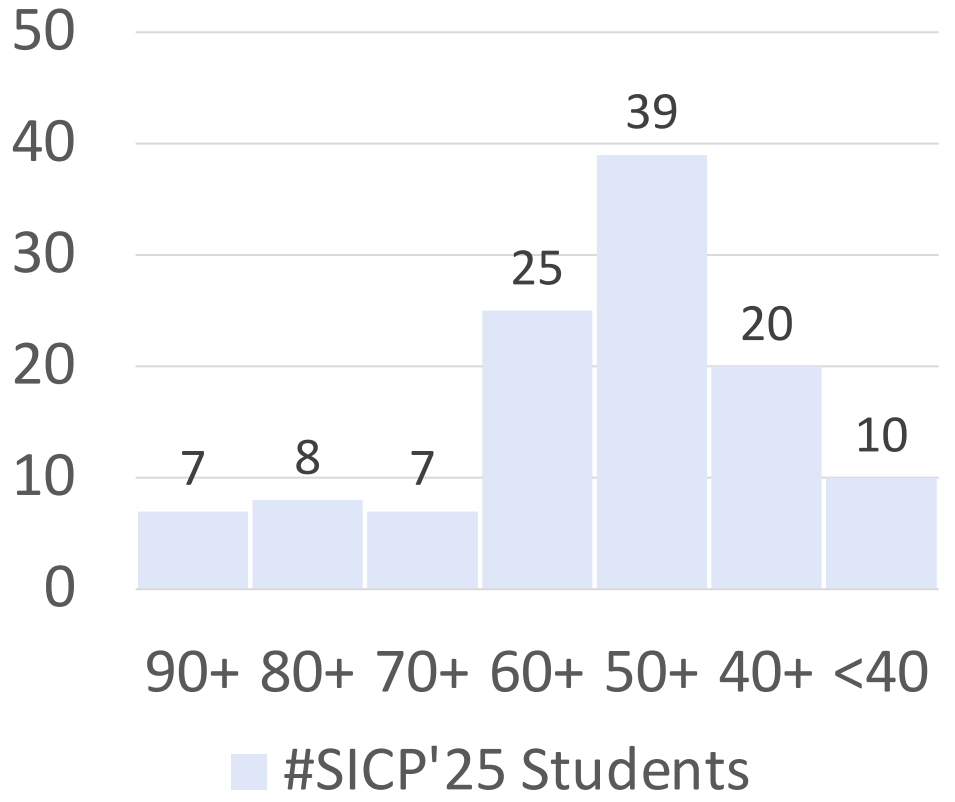
Whose Reflection?



We've evolved 2.44pts more evil this year?

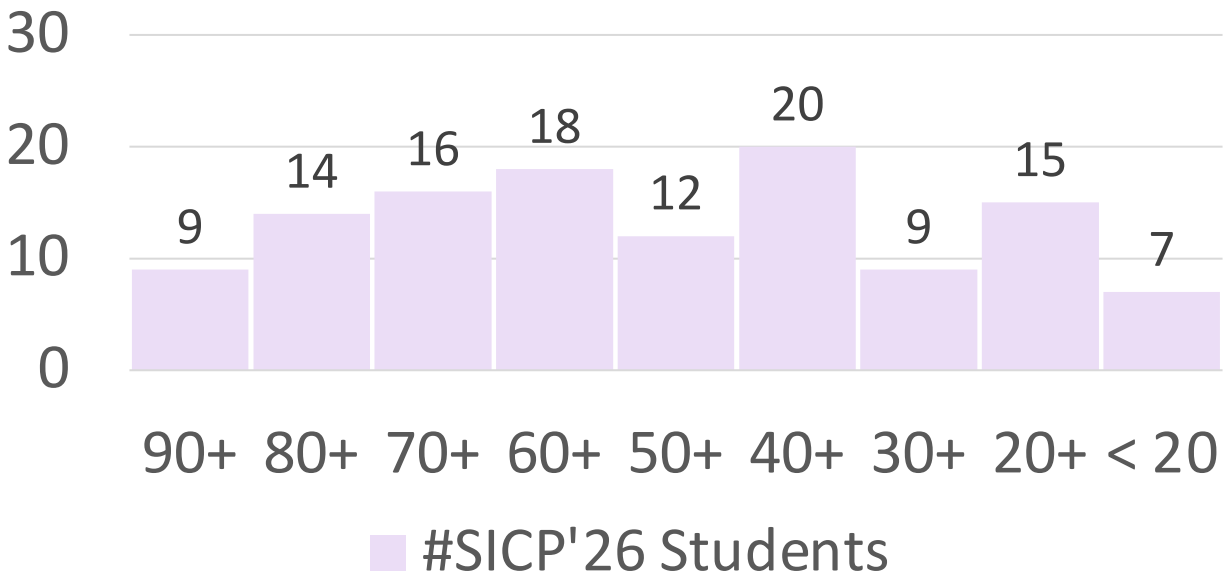


Maximum: 100 / **Average: 56.53** / Median: 57.5

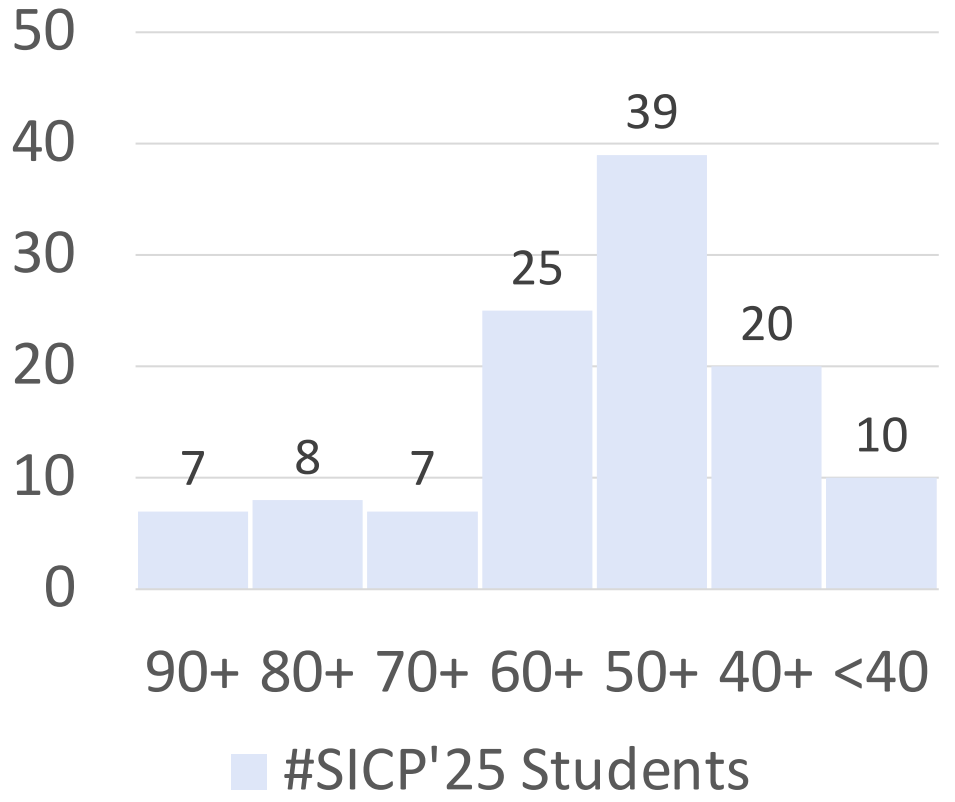


Maximum: 99 / **Average: 58.97** / Median: 57.5

We’ve evolved 2.44pts more evil this year?



Maximum: 100 / **Average: 56.53** / Median: 57.5



Maximum: 99 / **Average: 58.97** / Median: 57.5

Somehow, …

	P1	P2	P3	P4	P5	P6	P7	Bonus
Average	9.89	4.80	8.83	5.49	9.76	9.75	5.75	2.25
Total	15	10	15	10	15	20	15	3
Rate	66%	48%	59%	55%	65%	49%	38%	75%

Somehow, I unexpectedly saw ...

	P1	P2	P3	P4	P5	P6	P7	Bonus
Average	9.89	4.80	8.83	5.49	9.76	9.75	5.75	2.25
Total	15	10	15	10	15	20	15	3
Rate	66%	48%	59%	55%	65%	49%	38%	75%

WWPD + EnvDiagram?

WWPD + EnvDiagram

Jiacai Cui

PASCAL Group @ Nanjing University

WWPD: What Would Python Display

Treat yourself as the interpreter!

`print(print('Hello'), 'SICP')`	Hello （1分） None SICP （1分）

Unlike `return`, the `print` function is used to **print** values in the terminal. This can lead to some confusion between `print` and `return` because calling a function in the Python interpreter will print out the function's return value.

```
def what_is_printed():
    print('Hello World!')
    return 'Exiting this function.'
    print('SICP is awesome!')
```

```
>>> what_is_printed()
Hello World!
'Exiting this function.'
```

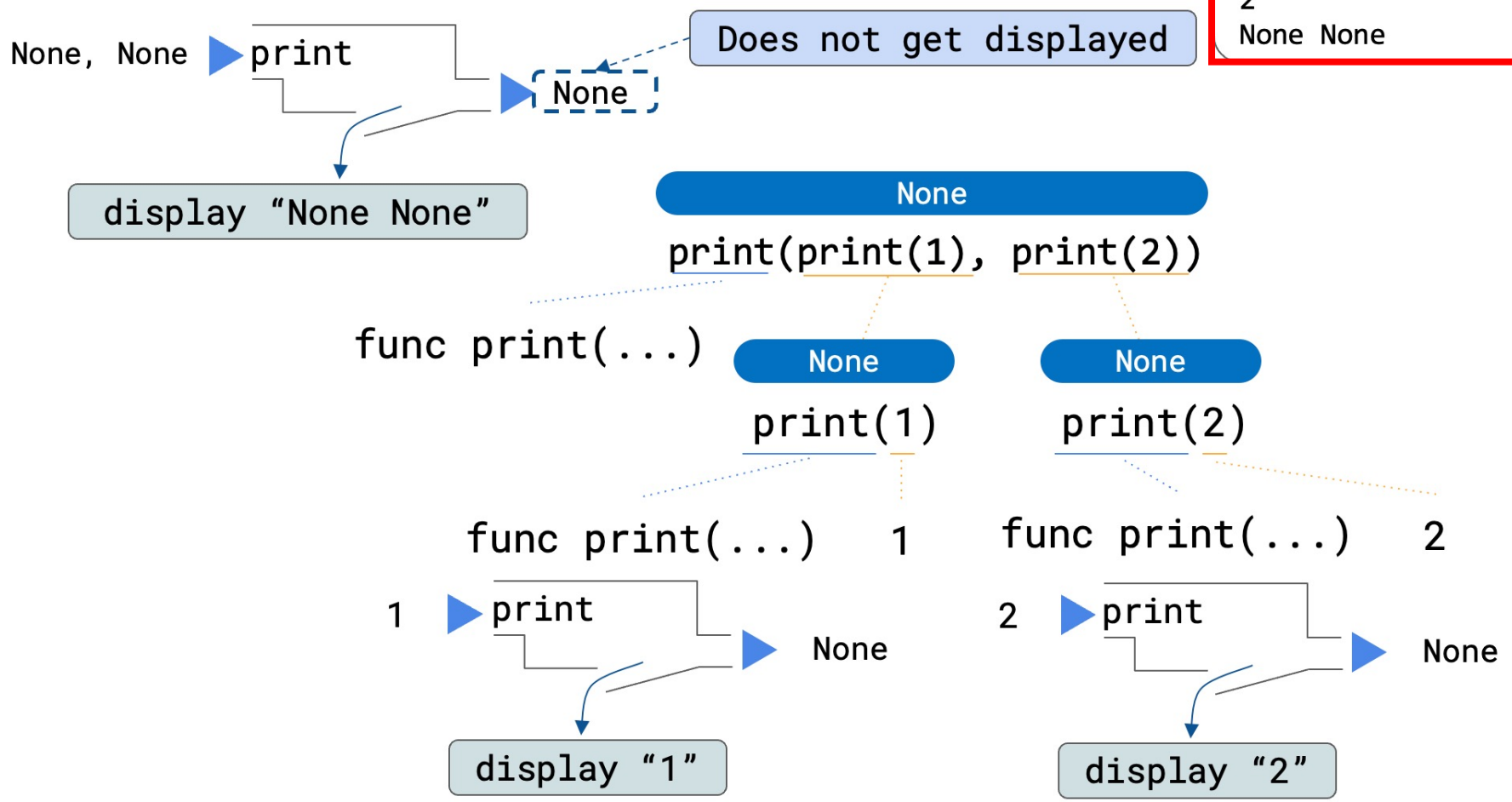
In the above example:

1. The first line of the output, `Hello World!`, is **printed** by the first `print` statement inside the function.
2. The second line of the output, `'Exiting this function.'`, is the **return** value that returned from the function `what_is_printed` to its call expression `what_is_printed()`, and then displayed by the Python interpreter.
3. The second print statement inside `what_is_printed` is never executed because the function has already terminated when the `return` statement is executed.
4. Also notice that `print` displays text without quotes, but `return` preserves them, as `what_is_printed` returns a string value.

(Lab01, Section 2.1.2, Link)

Nested Expressions with Print

```
>>> print(print(1), print(2))
1
2
None None
```



`print(False) and 1 / 0`	False （1分）

`f67 = lambda x: lambda y: (print(x + y) and 67)`
`f76 = lambda x: lambda y: (76 and print (x + 2*y))`
`f67(7)(6) or f76(6)(7)`
	13 （1分）
	20 （1分）

None Indicates that Nothing is Returned

- The special value **None** represents nothing in Python
- A function that does not explicitly return a value will return **None**
- *Careful:* **None** is *not displayed* by the interpreter as the value of an expression

```
>>> def does_not_return_square(x):
...     x * x
...
>>> does_not_return_square(4)
>>> sixteen = does_not_return_square(4)
>>> sixteen + 4
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

No return

None value is not displayed

The name sixteen is now bound to the value None

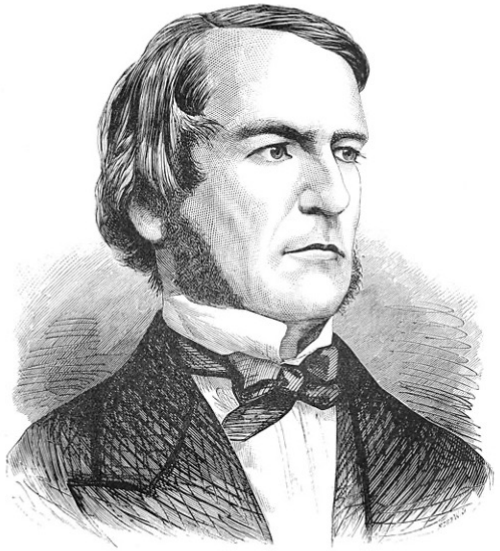
Boolean Expressions

Boolean expressions contain special operators **and**, **or**, **not**

- <exp1> **and** <exp2> **and** <exp3> **and** ...
  - Evaluate to the first false value.
  - If none are false, evaluates to the last expression
- <exp1> **or** <exp2> **or** <exp3> **or** ...
  - Evaluate to first true value.
  - If none are true, evaluates to the last expression
- **not** <exp>
  - Evaluates to True if <exp> if a *false value* and False if <exp> is a *true value*

(Sep 21, Control, Slide Page 13)

Boolean Contexts



George Boole

```
def absolute_value(x):
    """Return the absolute value of x."""
    if x < 0:
        return
    elif x == 0:
        return 0
    else:
        return x
```

Two boolean contexts

Boolean context is any place where an expression is evaluated to check if it's a True value or a False value

False values in Python: False, None, 0, '' (more to come)
True values: everything else

```
1  # Function
2
3  def make_cake(n):
4    def cake(x):
5      if x == 0:
6        return 'cake'
7      else:
8        print('The cake is a lie.')
9        n -= x
10     return cake(n)
11   return cake
```

make_cake(2) make_cake(2)(2)	Function（1分） The cake is a lie.（1分） Error（1分）

lab03: Recursion

```
>>> def f1(n):
...     def f2(x):
...         if x == 0:
...             return 'cake'
...         else:
...             print('The cake is a lie.')
...             n -= x
...             return f2(n)
...     return f2
>>> f1(2)
------

>>> f1(2)(2)
------
```

make_cake is f1;
cake is f2.

(Lab03, Question 1, Link)

However, we haven't been able to *modify* a variable in parent frames. Consider the following function:

```
def make_withdraw(balance):
    """Returns a function which can withdraw
    some amount from balance

    >>> withdraw = make_withdraw(50)
    >>> withdraw(25)
    25
    >>> withdraw(25)
    0
    """
    def withdraw(amount):
        if amount > balance:
            return "Insufficient funds"
        balance = balance - amount
        return balance
    return withdraw
```

The inner function `withdraw` attempts to update the variable balance in its parent frame. Running this function's doctests, we find that it causes the following error:

```
UnboundLocalError: local variable 'balance' referenced before assignment
```

Why does this happen? When we execute an assignment statement, remember that we are either creating a new binding in our current frame or we are updating an old one in the current frame. For example, the line `balance = ...` in `withdraw`, is creating the local variable balance inside `withdraw`'s frame. This assignment statement tells Python to expect a variable called balance inside `withdraw`'s frame, so Python will not look in parent frames for this variable. However, notice that we tried to compute `balance - amount` before the local variable was created! That's why we get the `UnboundLocalError`.

(Lab05, Section 2.1, Link)

`lst1 = [1, 2, 3, 4]`
`lst2 = lst1 + [5]`
`lst1.append(5)`
 	
`lst1 == lst2`
`lst1 is lst2`
	True（1分）
	False（1分）

Identity Operators

Identity

`<exp0> is <exp1>`

evaluates to True if both <exp0> and <exp1> evaluate to the same object

Equality

`<exp0> == <exp1>`

evaluates to True if both <exp0> and <exp1> evaluate to equal values

Identical objects are always equal values

(Oct 21, Mutable Values, Slide Page 74)

Each object has its own identity. That means you can construct two objects that may look exactly the same but have different identities.

```
>>> num1 = 257
>>> num2 = 257
>>> num1 == num2
True
>>> num1 is num2
False
>>> lst1 = [1, 2, 3, 4]
>>> lst2 = [1, 2, 3, 4]
>>> lst1 == lst2
True
>>> lst1 is lst2
False
```

Here, although the lists referred by lst1 and lst2 have exactly the same contents, they are not the same object. In other words, they are the same in terms of equality, but not in terms of identity (think about twins that look exactly the same but are in fact two different persons).

This is important in the discussion of mutability because when we mutate an object, we simply change its state, not its identity.

```
>>> lst1 = [1, 2, 3, 4]
>>> lst2 = lst1
>>> lst1.append(5)
>>> lst2
[1, 2, 3, 4, 5]
>>> lst1 is lst2
True
```

(Lab04, Section 2.5, Link)

Generators can Yield from Iterators

A **yield from** statement yields all values from an iterator or iterable (Python 3.3)

```
>>> list(a_then_b([3, 4], [5, 6]))
[3, 4, 5, 6]
```

```
def a_then_b(a, b):
    for x in a:
        yield x
    for x in b:
        yield x
```

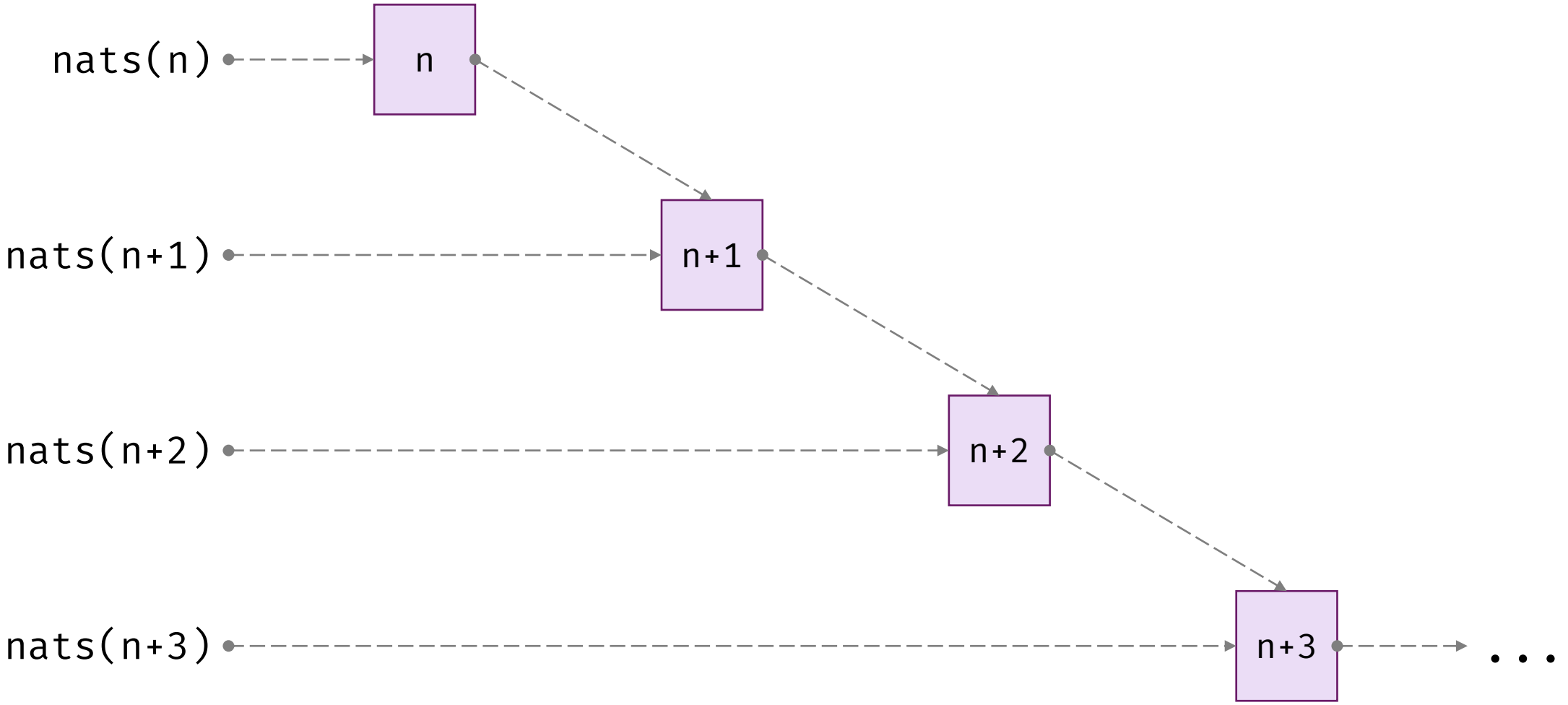
```
def a_then_b(a, b):
    yield from a
    yield from b
```

```
>>> list(countdown(5))
[5, 4, 3, 2, 1]
```

```
def countdown(k):
    if k > 0:
        yield k
        yield from countdown(k-1)
```

(Demo)

```
def nats(n):
    yield n
    yield from nats(n + 1)
```



Built-in Functions for Iteration

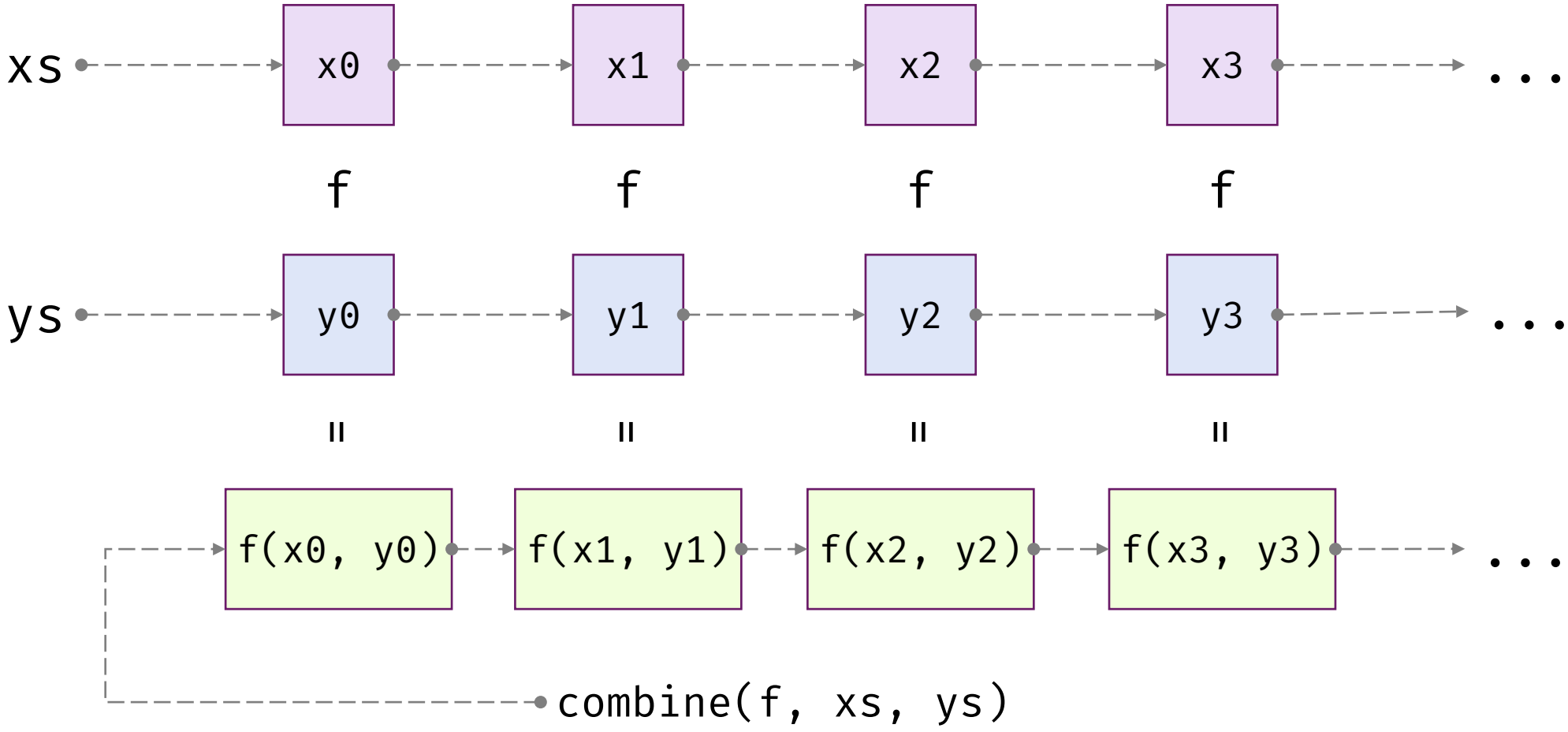
Many built-in Python sequence operations return iterators that compute results lazily

```
map(func, iterable):        Iterate over func(x) for x in iterable
filter(func, iterable):     Iterate over x in iterable if func(x)
zip(first_iter, second_iter):     Iterate over co-indexed (x, y) pairs
reversed(sequence):         Iterate over x in a sequence in reverse order
```

To view the contents of an iterator, place the resulting elements into a container

```
list(iterable):     Create a list containing all x in iterable
tuple(iterable):    Create a tuple containing all x in iterable
sorted(iterable):   Create a sorted list containing x in iterable
```

```
def combine(f, xs, ys):
    for x, y in zip(xs, ys):
        yield f(x, y)
```



```
evens = combine(add, nats(0), nats(0))
odds = combine(add, nats(0), nats(1))

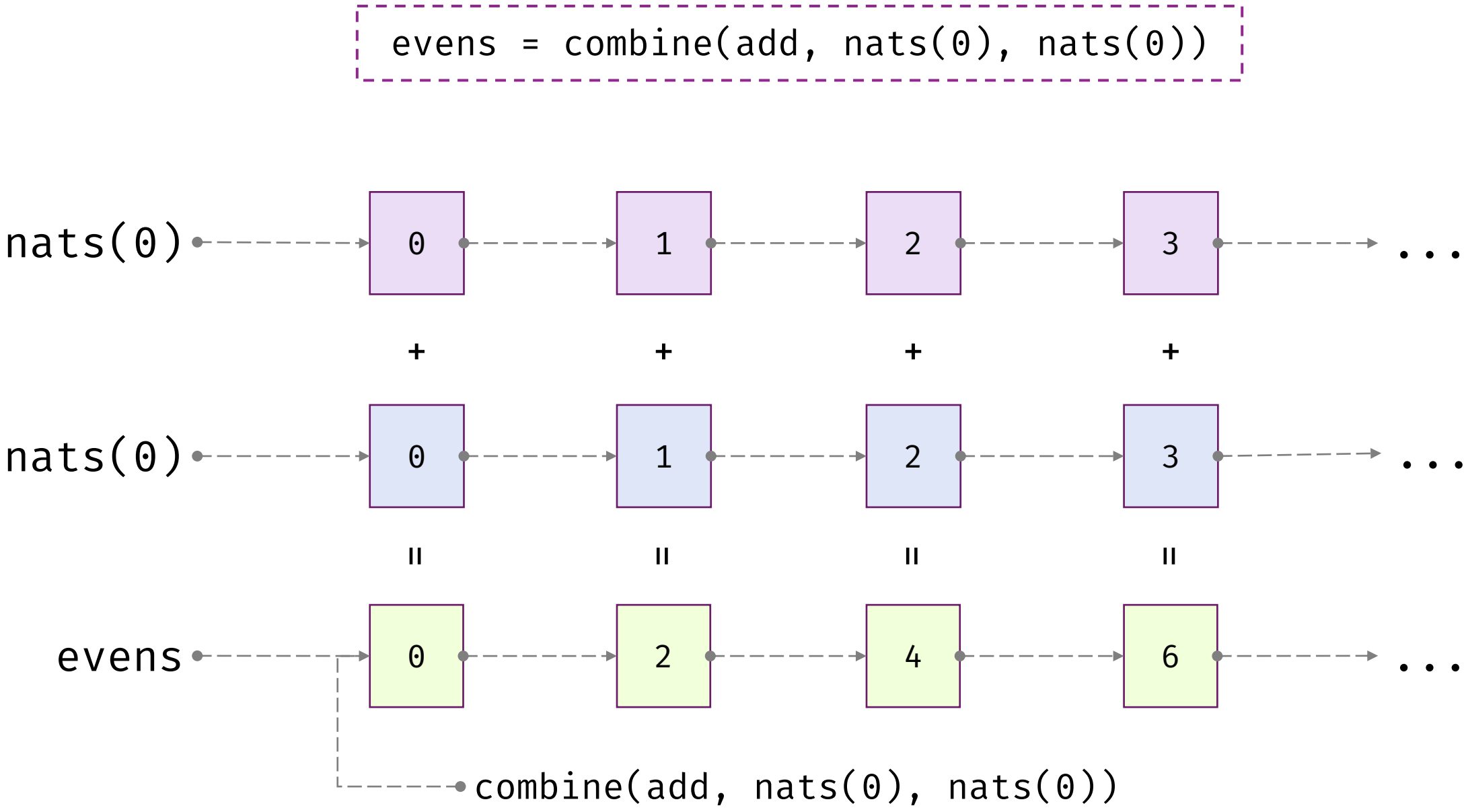
[next(evens) for _ in range(5)]
[next(odds) for _ in range(5)]

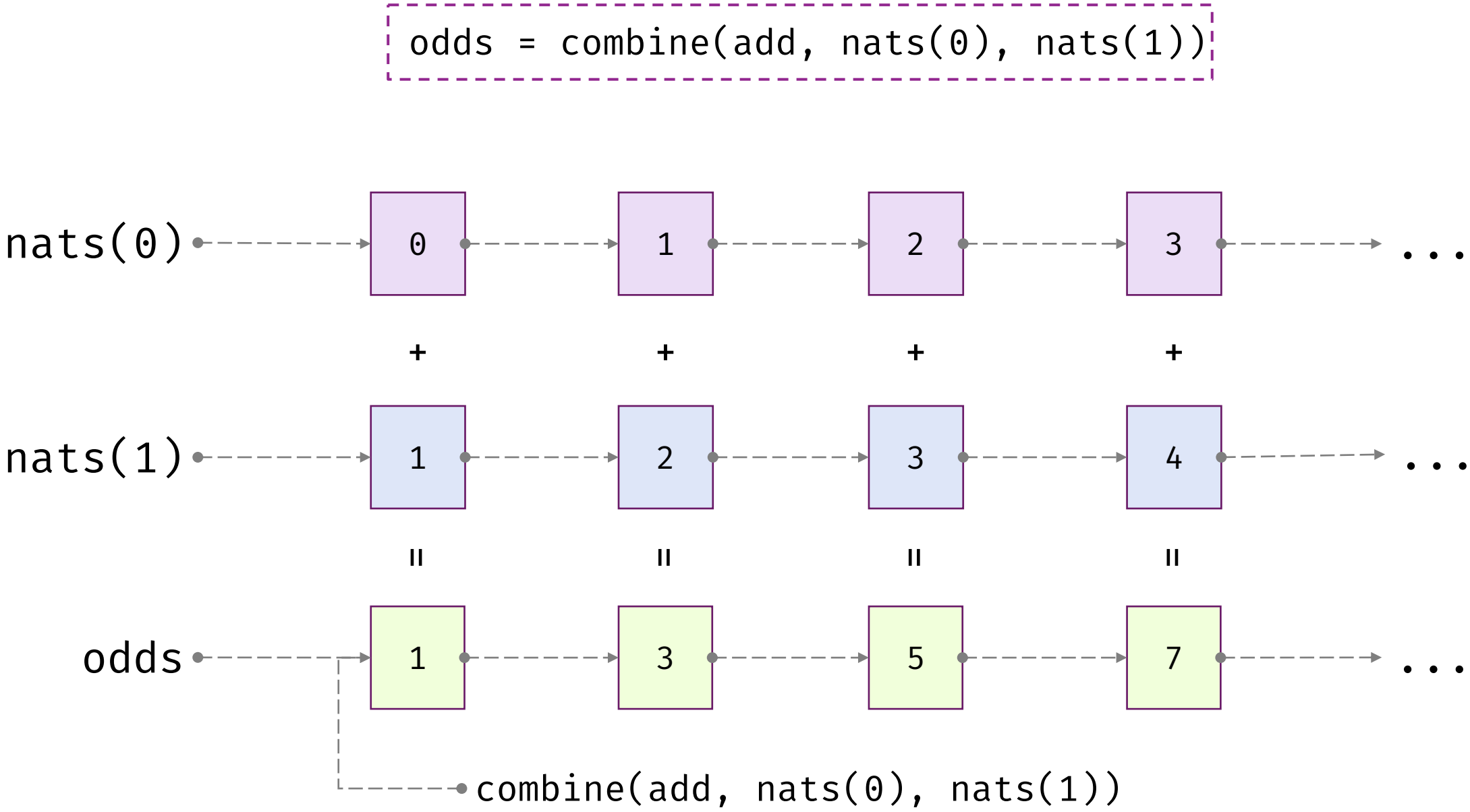
mystery = combine(add, odds, evens)
next(mystery)
```

[0, 2, 4, 6, 8] (1分)

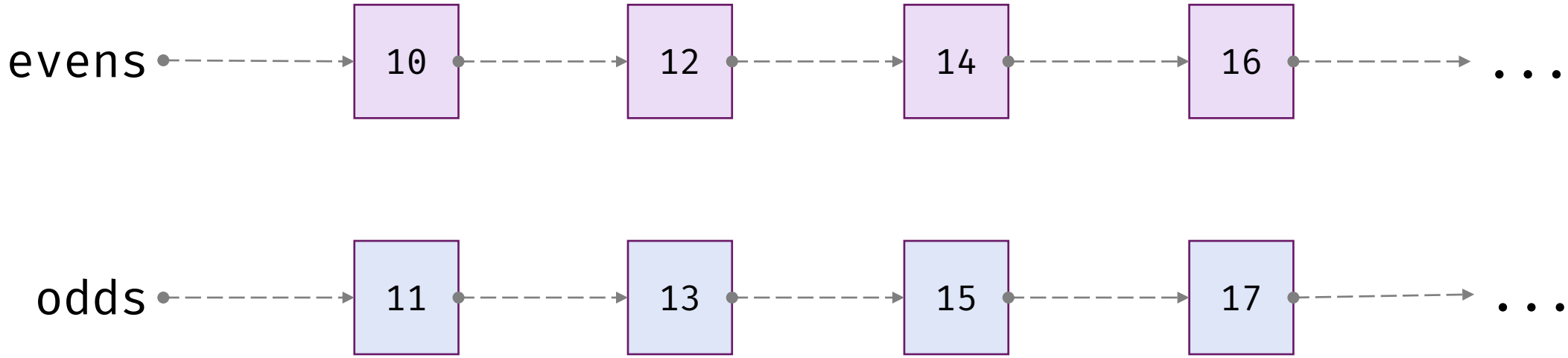
[1, 3, 5, 7, 9] (1分)

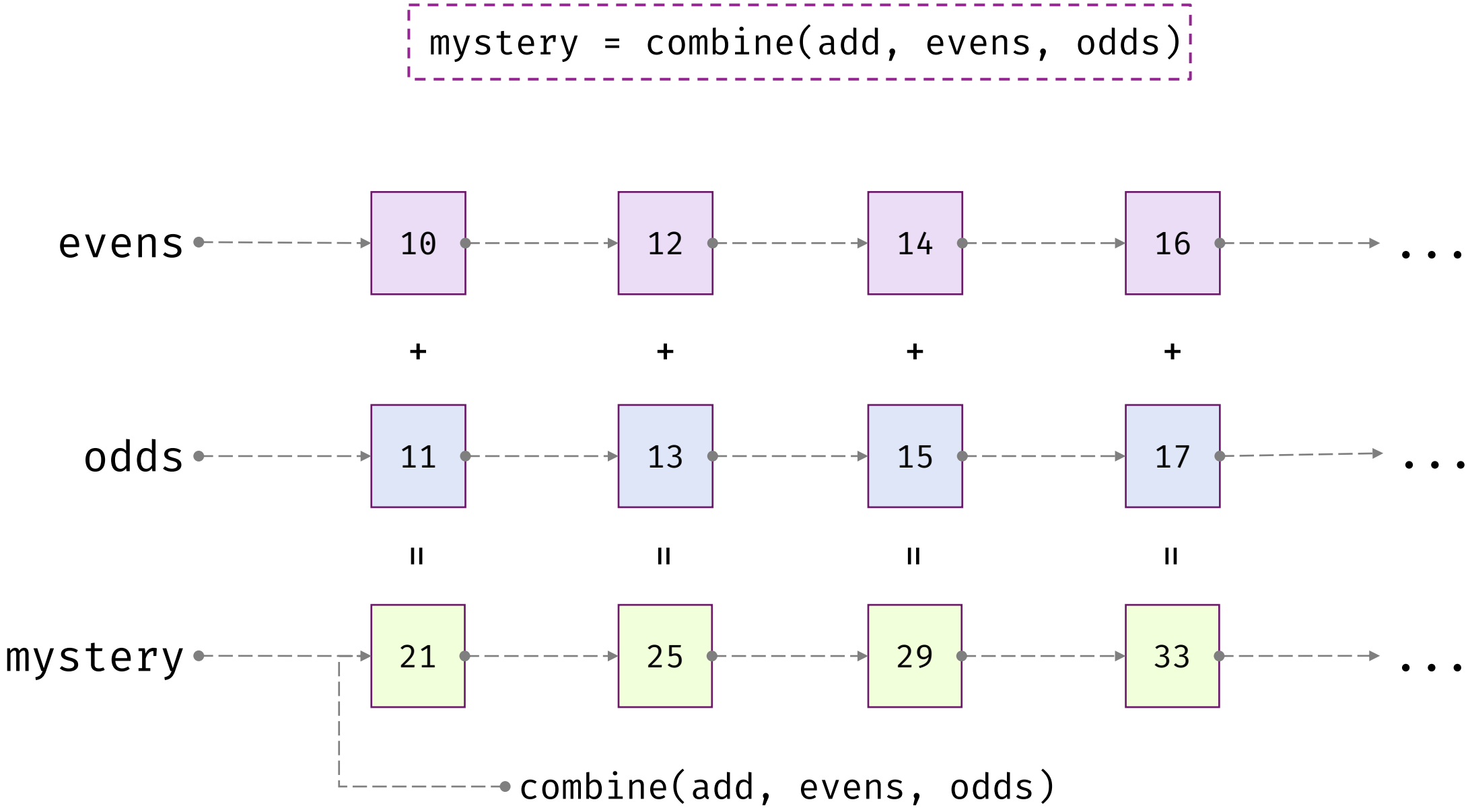
21 (1分)


evens = combine(add, nats(0), nats(0))
nats(0)
0
1
2
3
...
+
+
+
+
nats(0)
0
1
2
3
...
=
=
=
=
evens
0
2
4
6
...
combine(add, nats(0), nats(0))


odds = combine(add, nats(0), nats(1))
nats(0)
0
1
2
3
...
+
+
+
+
nats(1)
1
2
3
4
...
=
=
=
=
odds
1
3
5
7
...
combine(add, nats(0), nats(1))

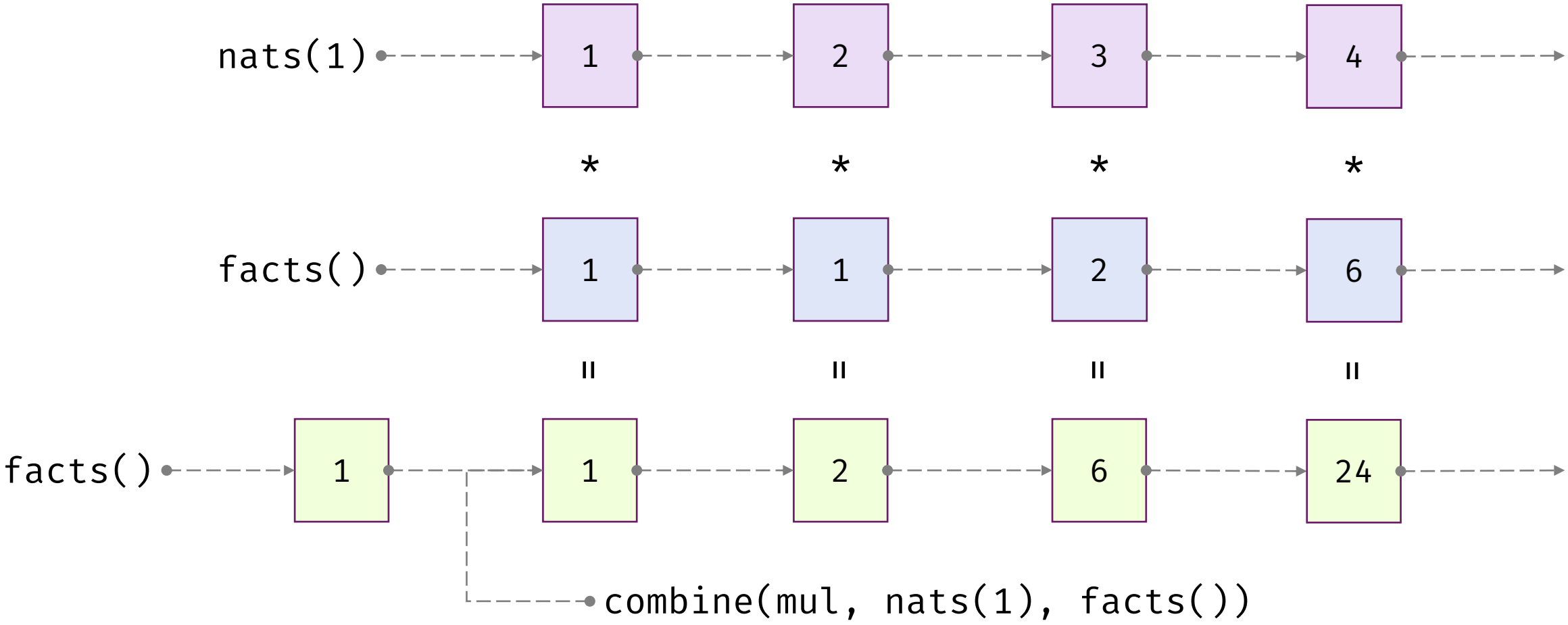
```
[next(evens) for _ in range(5)]
[next(odds) for _ in range(5)]
```



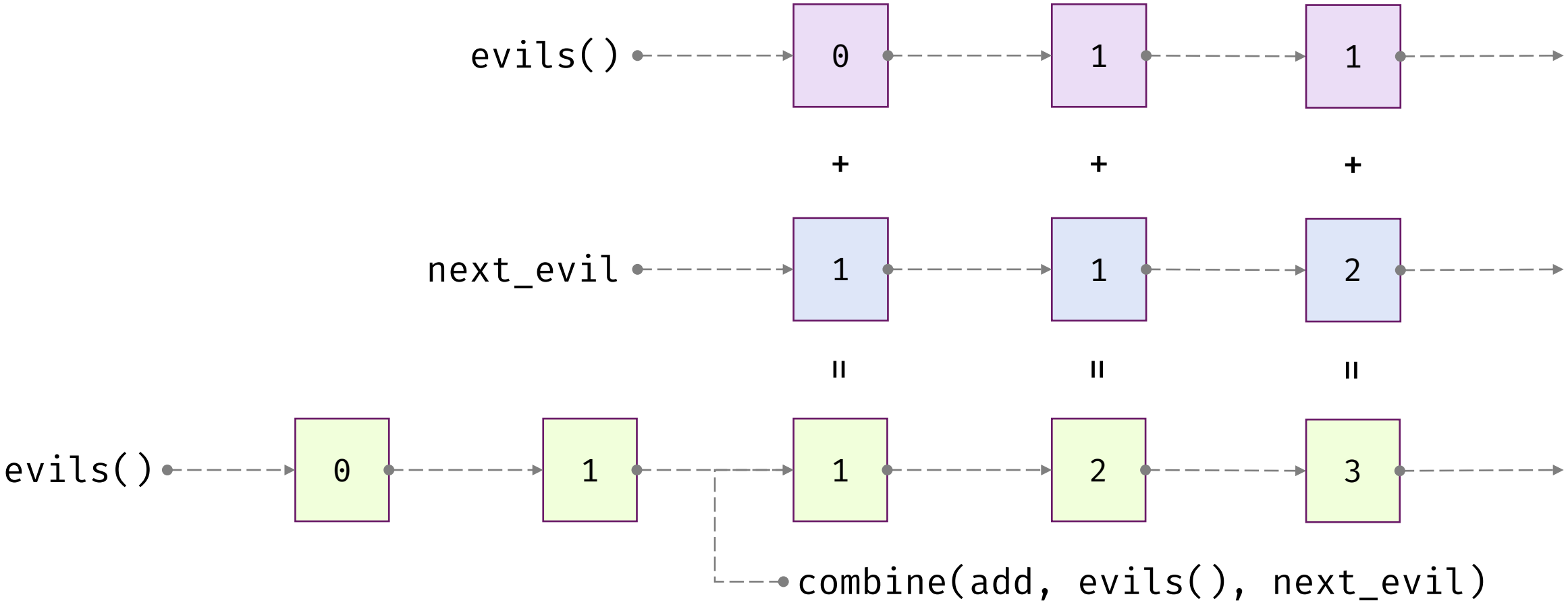

mystery = combine(add, evens, odds)
evens
10
12
14
16
...
+
+
+
+
odds
11
13
15
17
...
=
=
=
=
mystery
21
25
29
33
...
combine(add, evens, odds)

`my_fact = facts()` `[next(my_fact) for _ in range(5)]`	`[1, 1, 2, 6, 24]` （1分）
`my_evil = evils()` `[next(my_evil) for _ in range(7)]`	`[0, 1, 1, 2, 3, 5, 8]` （1分）

```
def facts():
    yield 1
    yield from combine(mul, nats(1), facts())
```

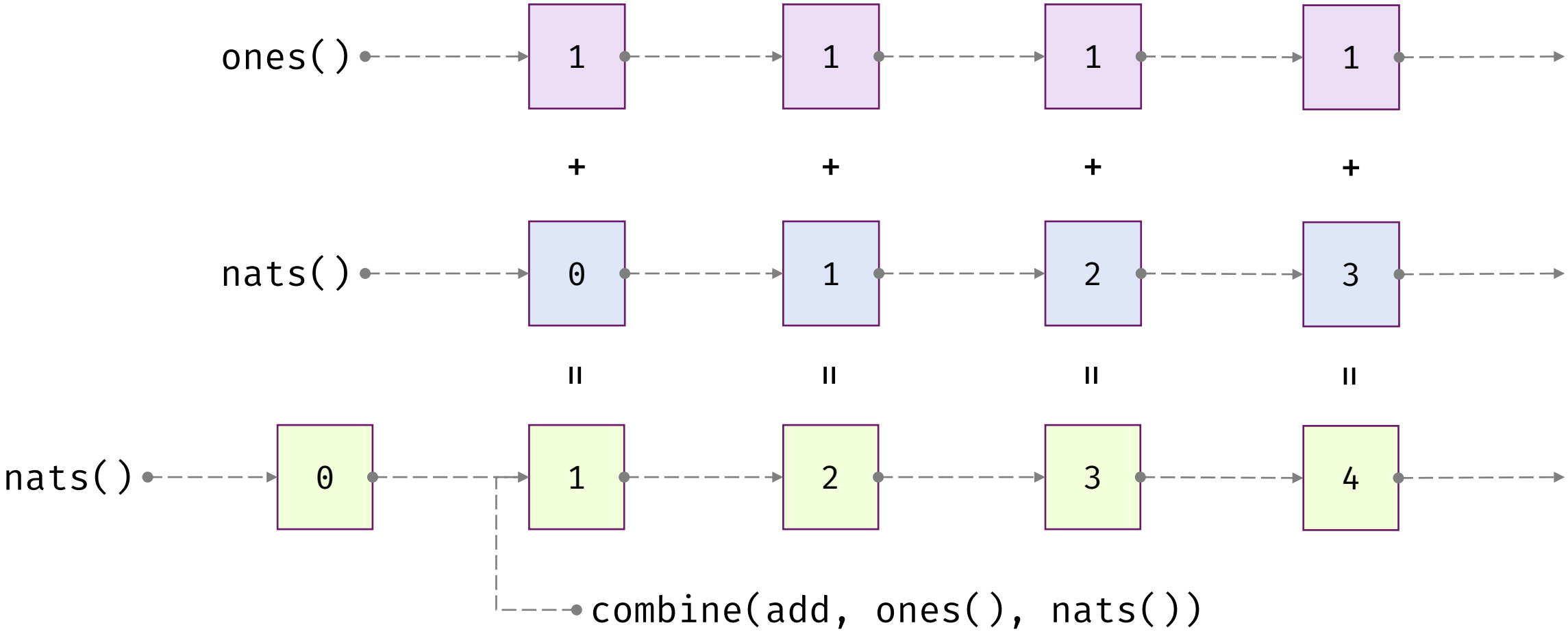


```
def evils():
    yield 0
    yield 1
    next_evil = evils()
    next(next_evil)
    yield from combine(add, evils(), next_evil)
```



```
def ones():
    yield 1
    yield from ones()
```

```
def nats():
    yield 0
    yield from combine(add, ones(), nats())
```

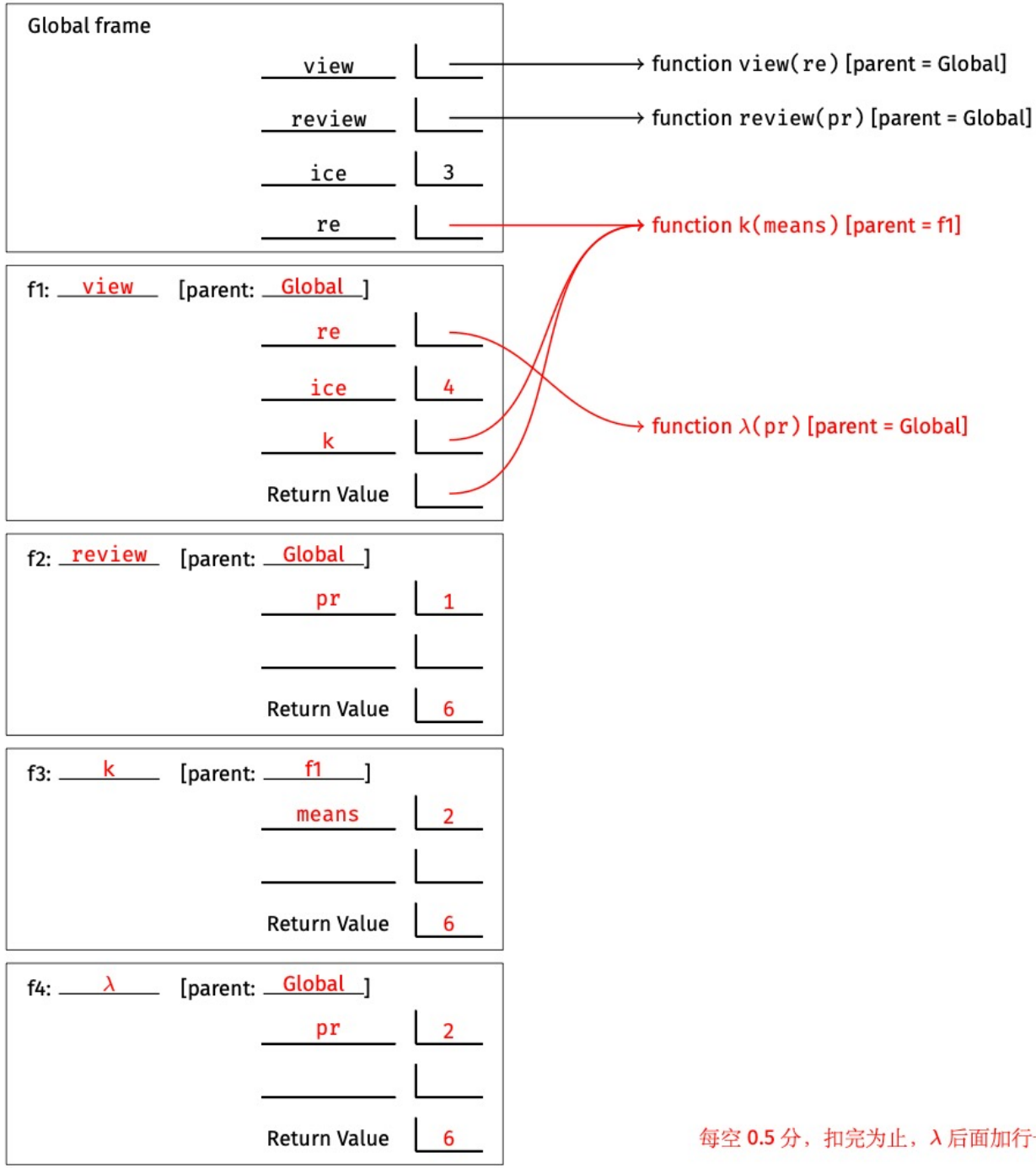


Environment Diagram

Treat yourself as the python tutor!

```
1  def view(re):
2      ice = 4
3      def k(means):
4          return re(means)
5      return k
6
7  def review(pr):
8      return re(pr * 2)
9
10 ice = 3
11 re = view(lambda pr: pr * ice)
12 review(1)
```

(Tutor Link)



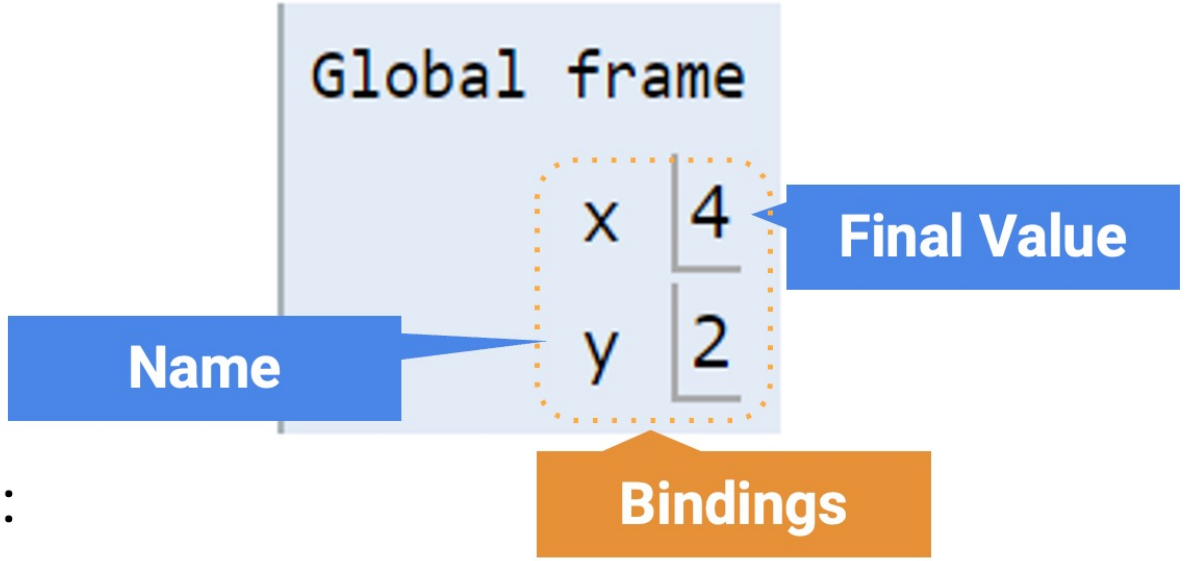
每空 0.5 分，扣完为止，λ 后面加行号不扣分。

Visualizing Assignment

Demo

Names are bound to **values** in an environment

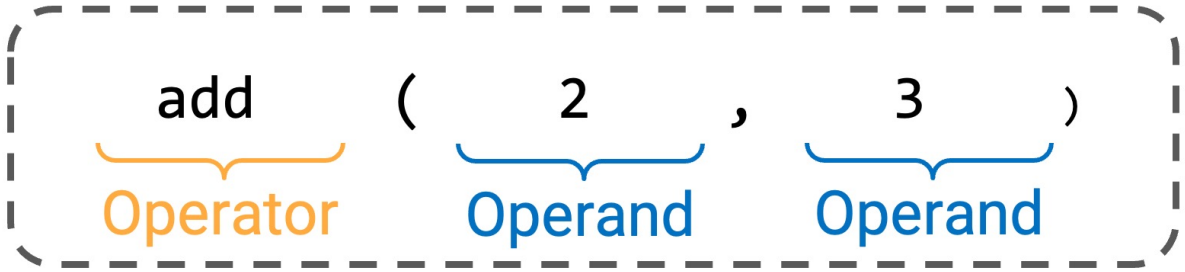
```
1  x = 1
2  y = 2
3  x = y * 2
```



To execute an assignment statement:

1. **Evaluate** the expression to the right of =.
2. **Bind** the value of the expression to the name to the left of = in the current environment.

Review - Evaluating Call Expressions



1. Evaluate
    a. Evaluate the operator subexpression
    b. Evaluate each operand subexpression
2. Apply
    a. Apply the value of the operator subexpression to the values of the operand subexpression

(Sep 16, Names & Functions, Slide Page 4)

How to draw an Environment Diagram

When a function is defined:

Create a function value:

```
    func <name>(<formal parameters>) [parent=<label>]
```

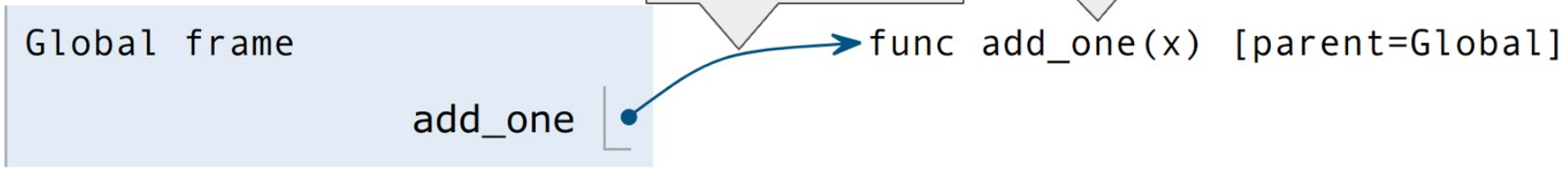
Its parent is the current frame.

Bind <name> to the function value in the current frame

```
def add_one(x):
    y = x + 1
```

Bind the name to the function value

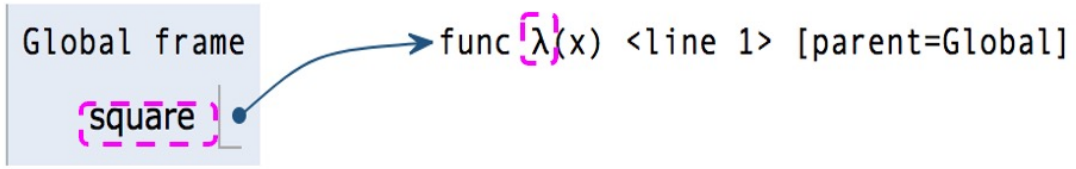
Create a function value



Lambda Expressions vs def Statements

```
square = lambda x: x * x
```

```
def square(x):
    return x * x
```





- Both create a function with the same behavior
- The parent frame of each function is the frame in which they were defined
- Both bind the function to the same name
- Only the def statement gives the function an intrinsic name

(Sep 23, Environment Diagrams, Slide Page 16)

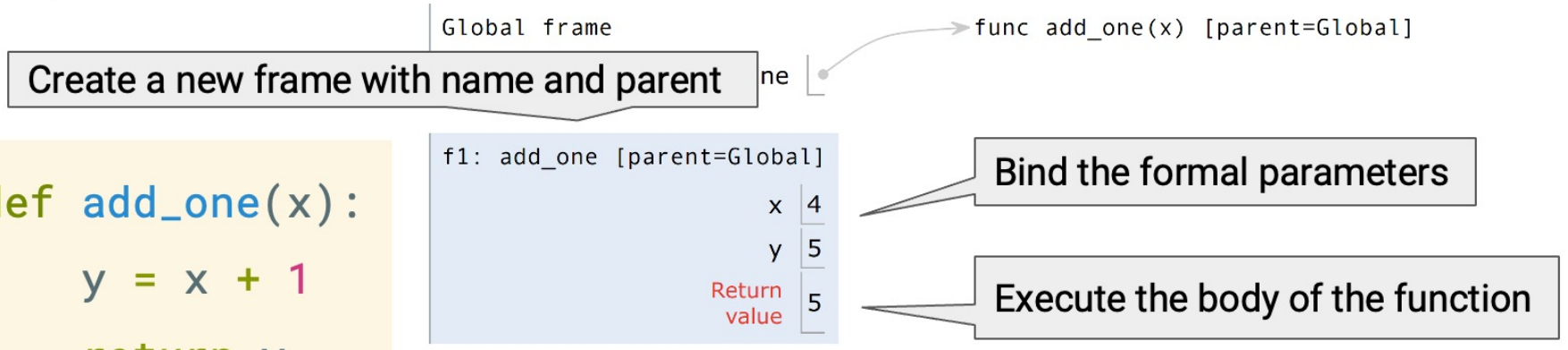
How to draw an Environment Diagram

Demo

When a function is applied:

1. Add a local frame, titled with the <name> of the function being applied.
2. Copy the parent of the **function** (not always the current frame) to the local frame: [parent=<label>]
3. Bind the <formal parameters> to the arguments in the local frame.
4. Execute the body of the function in the environment that starts with the local frame

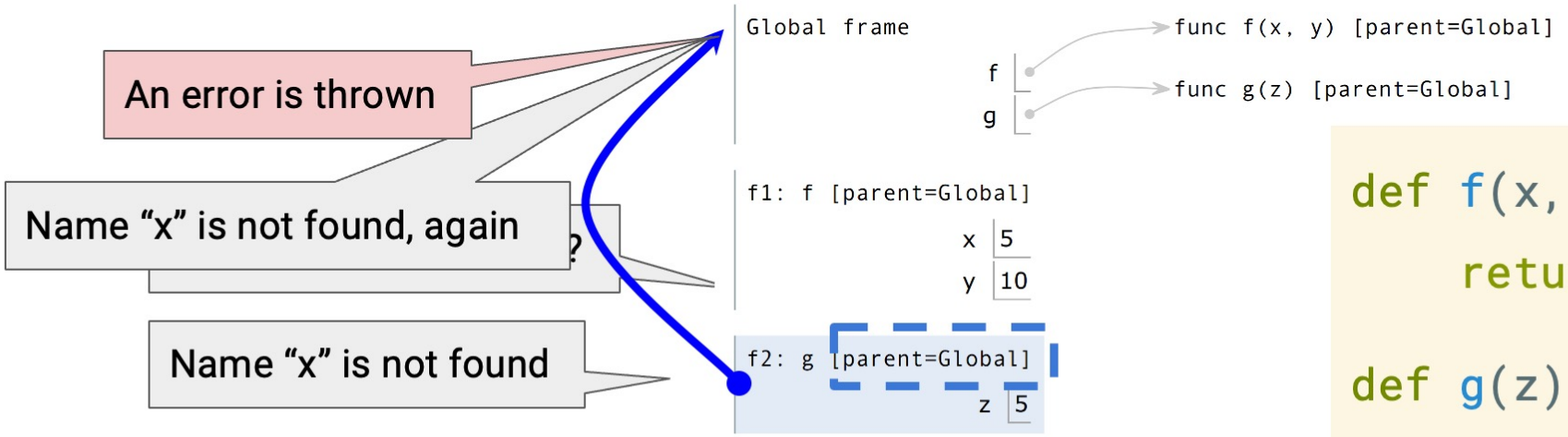
```
def add_one(x):
    y = x + 1
    return y
add_one(4)
```



Local Names

Variable Lookup:

- Lookup name in the current frame
- Lookup name in parent frame, its parent frame, etc..
- Stop at the global frame
- If not found, an error is thrown




```
def f(x, y):
    return g(x)

def g(z):
    return z + x

result = f(5, 10)
```

Important: There was no lookup done in f1 since the parent of f2 was Global

What happens here?

(Sep 23, Environment Diagrams, Slide Page 11)

Whose Reflection?

